

SRE STUDIUM PRZYPADKU

AZURE ARC HYBRID CLOUD LAB

Tygodnie 1-3: Kubernetes, Azure Arc, Terraform, CI/CD, Ingress
TLS, HPA, NetworkPolicy, RBAC

Autor: Bartosz Suszko

Firma: IT Solutions Bartosz Suszko, Olsztyn

Data: Maj 2026 (Tygodnie 1-3)

GitHub: github.com/buth11/sre-homelab-azure-arc

Stack: Kubernetes | Azure Arc | Terraform | KQL | Azure
DevOps | Traefik | HPA | NetworkPolicy | RBAC

Spis treści

1. Kontekst biznesowy i architektura
2. Infrastruktura On-Premise — klaster Kubernetes
3. FinOps — kontrola kosztów
4. Integracja hybrydowa — Azure Arc
5. Observability — Container Insights i Azure Monitor
6. Zapytania KQL — diagnostyka i SLO
7. Load Balancing — weryfikacja round-robin
8. Prometheus i Grafana
9. Infrastructure as Code — Terraform
10. Policy as Code — Azure Policy
11. GitHub — dokumentacja
12. CZESC 2: Service Principal, Remote State, CI/CD
13. CZESC 3: Ingress TLS, HPA, NetworkPolicy, RBAC
14. Postmortem — POST-001 etcd Split-Brain
15. Wyniki i wnioski
16. Troubleshooting Log (10 incydentów)

Rozdział 1: Kontekst i architektura

Projekt demonstruje budowę produkcyjnego środowiska hybrydowego na GMKtec EVO-X2 (128 GB RAM) łączącego Kubernetes z Microsoft Azure przez Azure Arc. Trzy tygodnie praktycznych ćwiczeń z dokumentacją każdego kroku i 10 udokumentowanych incydentów.

Warstwa	Technologia	Zastosowanie
Kubernetes	K3s v1.35.4+k3s1	Klaster 3 węzły
Ingress	Traefik + TLS	HTTPS routing po domenie
Autoscaling	HPA (autoscaling/v2)	Skalowanie CPU 2-8 replik
Network Security	NetworkPolicy	Firewall wewnątrz klastra
Access Control	RBAC + ServiceAccount	Least privilege
Cloud Bridge	Azure Arc	Hybrydowa platforma kontrolna
Monitoring	Azure Monitor + Prometheus	Observability
IaC	Terraform + Remote State	Infrastructure as Code
CI/CD	Azure DevOps	Pipeline z approval gate

Rozdział 2: Infrastruktura On-Premise

Wezel	IP	CPU	RAM
k3s-master (Control Plane)	192.168.122.10	4 vCPU	8 GB
k3s-worker1	192.168.122.11	4 vCPU	16 GB
k3s-worker2	192.168.122.12	4 vCPU	16 GB

Rozdział 3: FinOps

The screenshot displays the Microsoft Azure Cost Management dashboard for user Bartosz Suszko. The interface is in Polish. The left sidebar contains navigation links for various Azure services. The main content area is titled 'Cost Management: Bartosz Suszko | Budżety' (Budgets). It shows a table with one budget entry: 'SRE-Lab-Budget' with a monthly budget of 5.00 USD and a current cost of 0.00 USD. The table columns include Name, Scope, Reset period, Creation date, Expiration date, Budget, Forecast, Estimated cost, and Progress. The progress bar for the budget is at 0.00%.

Nazwa	Zakres	Resetuj okres	Data utworzenia	Data wygaśnięcia	Budżet	Prognozowane	Oszacowane wy...	Postęp
SRE-Lab-Budget	55df28b8-39ed-58b...	Monthly	1.05.2026	30.04.2028	5,00 USD		\$0.00	0.00%

Zrzut 1: Azure Cost Management — budżet 5 USD/mies., koszt: 0.00 USD.

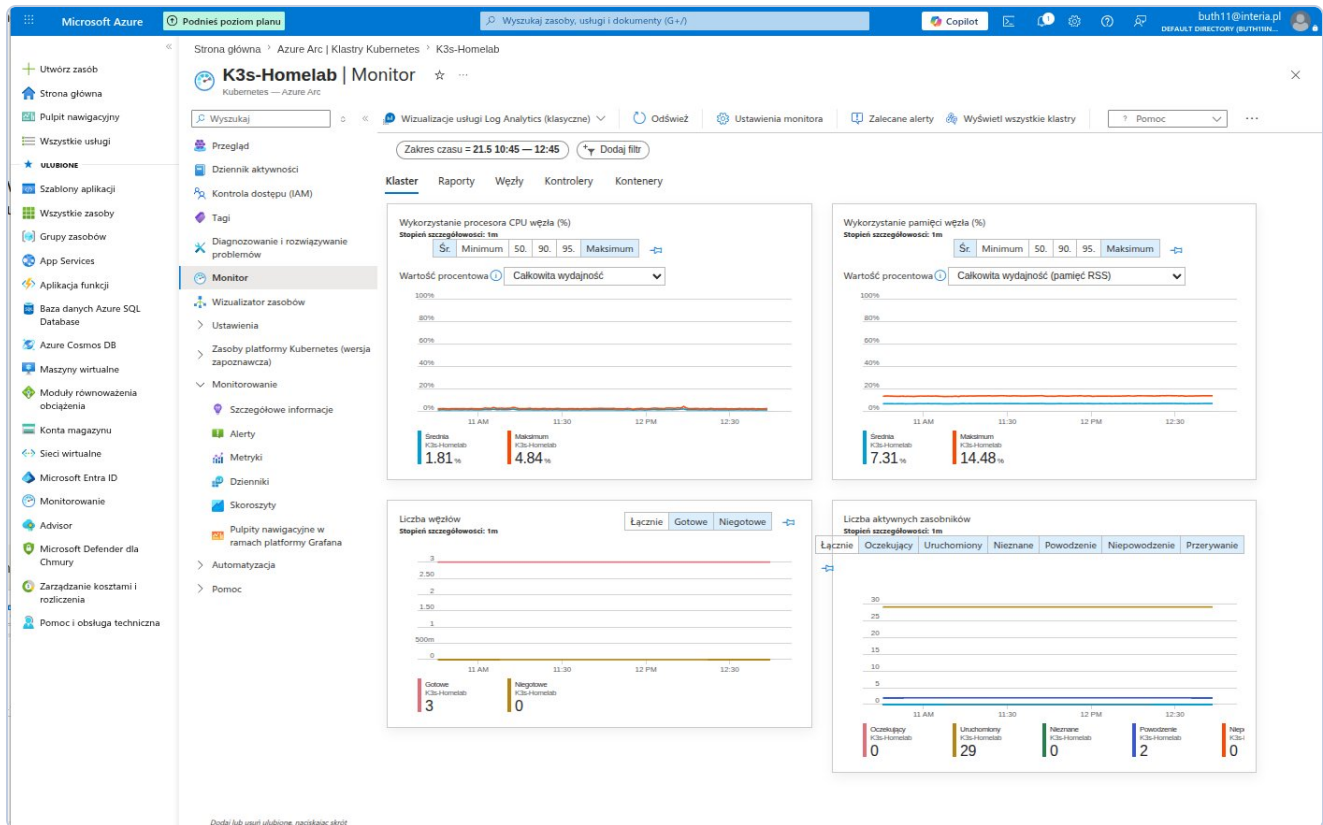
Rozdział 4: Azure Arc

Zrzut 2: Azure Arc — K3s-Homelab Connected, v1.35.4+k3s1, 3 węzły, 12 rdzeni.

Rozdział 5: Observability

```
Desktop: bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × Desktop: bash × (buth11) 192.168.122.10 ×
{
  "principalId": "a3fe4a8c-7a2e-4b99-8de9-2150d617cb76",
  "tenantId": null,
  "type": "SystemAssigned"
},
"isSystemExtension": false,
"name": "azuremonitor-containers",
"packageUri": null,
"plan": null,
"provisioningState": "Succeeded",
"releaseTrain": "Stable",
"resourceGroup": "SRE-Lab-RG",
"scope": {
  "cluster": {
    "releaseNamespace": "azuremonitor-containers"
  },
  "namespace": null
},
"statuses": [],
"systemData": {
  "createdAt": "2026-05-22T07:44:41.936626+00:00",
  "createdBy": null,
  "createdByType": null,
  "lastModifiedAt": "2026-05-22T07:44:41.936626+00:00",
  "lastModifiedBy": null,
  "lastModifiedByType": null
},
"type": "Microsoft.KubernetesConfiguration/extensions",
"version": null
}
buth11@k3s-master:~$ sudo k3s kubectl get pods -n kube-system | grep ama
[sudo] password for buth11:
ama-logs-9m9x9          3/3      Running    0          37s
ama-logs-9scpr          3/3      Running    0          37s
ama-logs-rs-65b58c7f96-f8lds 2/2      Running    0          37s
ama-logs-spcdp          3/3      Running    0          37s
buth11@k3s-master:~$
```

Zrzut 3: Pody AMA Running — 4 agenty, 0 restartow.



Zrzut 4: Azure Monitor — CPU avg 1.58%, RAM avg 6.27%, 3 wezly Ready, 29 podow.

Rozdział 6: KQL

The screenshot displays the Microsoft Azure portal interface. On the left is the navigation pane with 'Log Analytics' selected. The main area shows the 'Logs' section of a workspace named 'DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU'. A new KQL query is entered and executed, showing results for 'KubeNodeInventory'.

Query:

```
1 KubeNodeInventory
2 where TimeGenerated > ago(1h)
3 summarize arg_max(TimeGenerated, *) by Computer
4 project
5   Computer,
6   Status,
7   KubeletVersion
8 order by Computer asc
```

Results:

Computer	Status	KubeletVersion
k3s-master	Ready	v1.35.4+k3s1
k3s-worker1	Ready	v1.35.4+k3s1
k3s-worker2	Ready	v1.35.4+k3s1

KQL-1: 3 nodes Ready, v1.35.4+k3s1.

DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU | Logs

Log Analytics workspace

Search

New Query 1* ... +

Time range: Set in query Show: 1000 results KQL mode

```

1 KubePodInventory
2 where TimeGenerated > ago(1h)
3 summarize arg_max(TimeGenerated, *) by Name, Namespace
4 summarize PodCount = count() by Namespace, PodStatus
5 order by Namespace asc, PodCount desc

```

Namespace	PodStatus	PodCount
azure-arc	Running	12
default	Running	3
kube-system	Running	14
kube-system	Succeeded	2

0s 533ms Display time (UTC+00:00) Query details 1 - 4 of 4

KQL-2: azure-arc 12 Running, default 3 Running, kube-system 14+2.

DefaultWorkspace-b023e8b8-4446-4d4f-9fcf-d20a6c1fa093-WEU | Logs

Log Analytics workspace

Search

New Query 1* ... +

Time range: Set in query Show: 1000 results KQL mode

```

1 KubeEvents
2 where TimeGenerated > ago(24h)
3 project TimeGenerated, Namespace, Name, Reason, Message
4 order by TimeGenerated desc
5 take 20

```

TimeGenerated [UTC]	Namespace	Name	Reason
5/22/2026, 7:44:56.000 AM	kube-system	ama-logs-9cpr	FailedMount
5/22/2026, 7:44:56.000 AM	kube-system	ama-logs-rs-65b58c7f96-f8ld	FailedMount
5/22/2026, 7:35:32.000 AM	azure-arc	kube-aad-proxy-5f59465457-12...	Unhealthy
5/22/2026, 7:35:25.000 AM	azure-arc	config-agent-7548f59c5-k4zrv	Unhealthy
5/22/2026, 7:35:13.000 AM	k3s-worker2		Rebooted
5/22/2026, 7:35:11.000 AM	k3s-worker1		Rebooted
5/22/2026, 7:35:09.000 AM	kube-system	metrics-server-786d997795-zz...	Unhealthy
5/22/2026, 7:35:07.000 AM	k3s-master		Rebooted
5/21/2026, 10:53:39.000 AM	kube-system	ama-logs-89tpj	Unhealthy
5/21/2026, 10:52:38.000 AM	kube-system	ama-logs-rs-65b58c7f96-nkshw	Unhealthy
5/21/2026, 8:37:39.000 AM	kube-system	svclb-aras-demo-app-6f6f7352...	FailedScheduling
5/21/2026, 8:37:39.000 AM	kube-system	svclb-aras-demo-app-6f6f7352...	FailedScheduling
5/21/2026, 8:37:39.000 AM	kube-system	svclb-aras-demo-app-6f6f7352...	FailedScheduling

0s 287ms Display time (UTC+00:00) Query details 1 - 13 of 13

KQL-3: 13 eventow 24h — Rebooted, Unhealthy (chwilowe), FailedScheduling.

The screenshot shows the Microsoft Azure Log Analytics interface. The left sidebar contains navigation options like Home, Dashboard, All services, and Favorites. The main area displays a Log Analytics workspace with a search bar and a list of logs. A KQL query is entered in the 'New Query 1' field:

```
1 KubePodInventory
2 | where TimeGenerated > ago(24h)
3 | summarize arg_max(TimeGenerated, *) by Name, Namespace
4 | where PodRestartCount > 0
5 | project Namespace, Name, PodRestartCount, PodStatus
6 | order by PodRestartCount desc
```

The results table shows the following data:

Namespace	Name	PodRestartCount	PodStatus
azure-arc	extension-manager-85899588c...	3	Running
azure-arc	clusterconnect-agent-5bc68597...	3	Running
kube-system	svcib-traefik-4b7f5cb6-8b8x...	2	Running
azure-arc	extension-events-collector-6c9...	2	Running
azure-arc	clusteridentityoperator-767768...	2	Running
azure-arc	kube-aad-proxy-5f5465457-42...	2	Running
kube-system	svcib-traefik-4b7f5cb6-c2gs...	2	Running
azure-arc	config-agent-75548f59c5-k4zrv...	2	Running
azure-arc	cluster-metadata-operator-6db...	2	Running
kube-system	svcib-traefik-4b7f5cb6-9f587...	2	Running
azure-arc	metrics-agent-6d994764-rc44...	2	Running
azure-arc	controller-manager-7cb948ddff...	2	Running
azure-arc	resource-sync-agent-5f4688cc5...	2	Running
azure-arc	flux-logs-agent-76f8c454c9-9g...	1	Running
azure-arc	logcollector-8bd4468b4-4s2zw...	1	Running
kube-system	ama-logs-nlg5p	1	Running
kube-system	helm-install-traefik-ggts...	1	Succeeded
kube-system	svcib-aras-demo-app-8581d79...	1	Running
default	aras-demo-app-f78659546-vb...	1	Running
kube-system	coredns-c4dbfbb5f-llkx...	1	Running
kube-system	local-path-provisioner-5c4dc5d...	1	Running
kube-system	metrics-server-786d997795-zz...	1	Running
kube-system	svcib-aras-demo-app-8581d79...	1	Running

KQL-4: 27 podow z restartami po restarcie VM — wszystkie Running.

The screenshot shows the Microsoft Azure Log Analytics interface. The left sidebar contains navigation options like Home, Dashboard, All services, and Favorites. The main area displays a Log Analytics workspace with a search bar and a list of logs. A KQL query is entered in the 'New Query 1' field:

```
1 KubeNodeInventory
2 | where TimeGenerated > ago(24h)
3 | summarize
4 |   TotalSamples = count(),
5 |   ReadySamples = countif(Status == "Ready")
6 | by Computer
7 | extend AvailabilityPct = round((toReal(ReadySamples) / toReal(TotalSamples)) * 100, 2)
8 | project Computer, AvailabilityPct, ReadySamples, TotalSamples
9 | order by Computer asc
```

The results table shows the following data:

Computer	AvailabilityPct	ReadySamples	TotalSamples
k3s-master	100	184	184
k3s-worker1	100	184	184
k3s-worker2	100	184	184

KQL-5: SLO 100% — 184/184 probek Ready.

SLO 100% w oknie 24h. Error budget nienaruszony nawet po pelnym restarcie VM.

Rozdział 7: Load Balancing

```
Desktop: bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × Desktop: bash × (buth11) 192.168.122.10 ×
https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

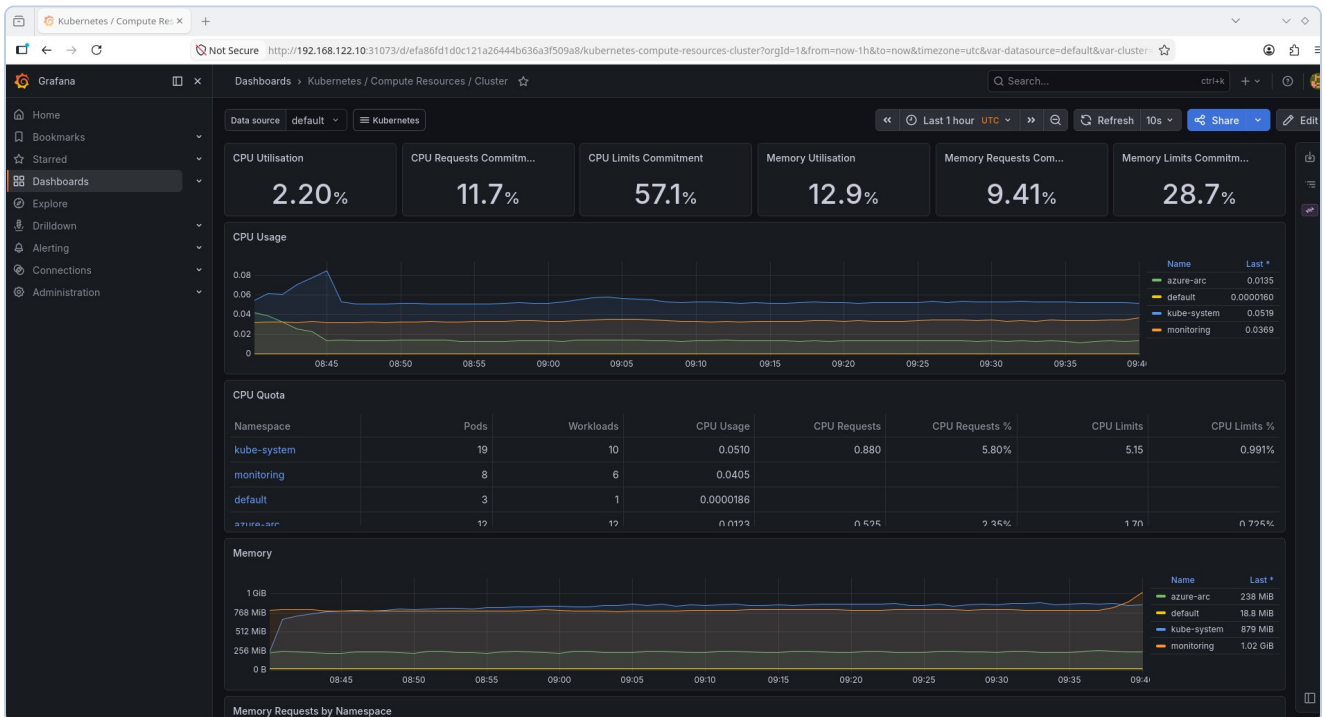
0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Fri May 22 07:42:36 2026 from 192.168.122.1
buth11@k3s-master:~$ for i in {1..20}; do
  curl -s http://192.168.122.10:8080 | grep "Hostname"
  sleep 0.5
done
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-vbwkm
Hostname: aras-demo-app-f78659546-vbwkm
Hostname: aras-demo-app-f78659546-vbwkm
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-vbwkm
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-4lprm
buth11@k3s-master:~$
```

Zrzut 5: Round-robin — 20 requestow na 3 pody, 900 rownolegla bez bledow.

Rozdział 8: Prometheus & Grafana



Rzrzut 6: Grafana — CPU 2.20%, RAM 12.9%, CPU Quota per namespace.

Rozdział 9: Terraform IaC

```
Desktop : bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × terraform : bash × (buth11) 192.168.122.10 ×
Plan: 6 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ action_group_id           = (known after apply)
+ budget_name               = "SRE-Lab-Budget"
+ log_analytics_workspace_id = (known after apply)
+ log_analytics_workspace_key = (sensitive value)
+ resource_group_id         = (known after apply)
+ resource_group_name       = "SRE-Lab-RG"
+ subscription_id           = "b023e8b8-4446-4d4f-9fcf-d20a6c1fa093"

Saved the plan to: tfplan

To perform exactly these actions, run the following command to apply:
  terraform apply "tfplan"
buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$ terraform apply tfplan
random_string.suffix: Creating...
random_string.suffix: Creation complete after 0s [id=5plwbb]
azurerm_resource_group.sre_lab: Creating...
azurerm_consumption_budget_subscription.sre_lab: Creating...
azurerm_consumption_budget_subscription.sre_lab: Still creating... [00m10s elapsed]
azurerm_consumption_budget_subscription.sre_lab: Creation complete after 19s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/providers/Microsoft.Consumption/budgets/SRE-Lab-Budget]

Error: A resource with the ID "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG" already exists -
to be managed via Terraform this resource needs to be imported into the State. Please see the resource documentation for "azure
rm_resource_group" for more information.

with azurerm_resource_group.sre_lab,
on main.tf line 26, in resource "azurerm_resource_group" "sre_lab":
26: resource "azurerm_resource_group" "sre_lab" {

buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$
```

Terraform Plan: 6 zasobow, 0 zmian.

```
Desktop: bash × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × (buth11) 192.168.122.10 × terraform : bash × (buth11) 192.168.122.10 ×

Saved the plan to: tfplan

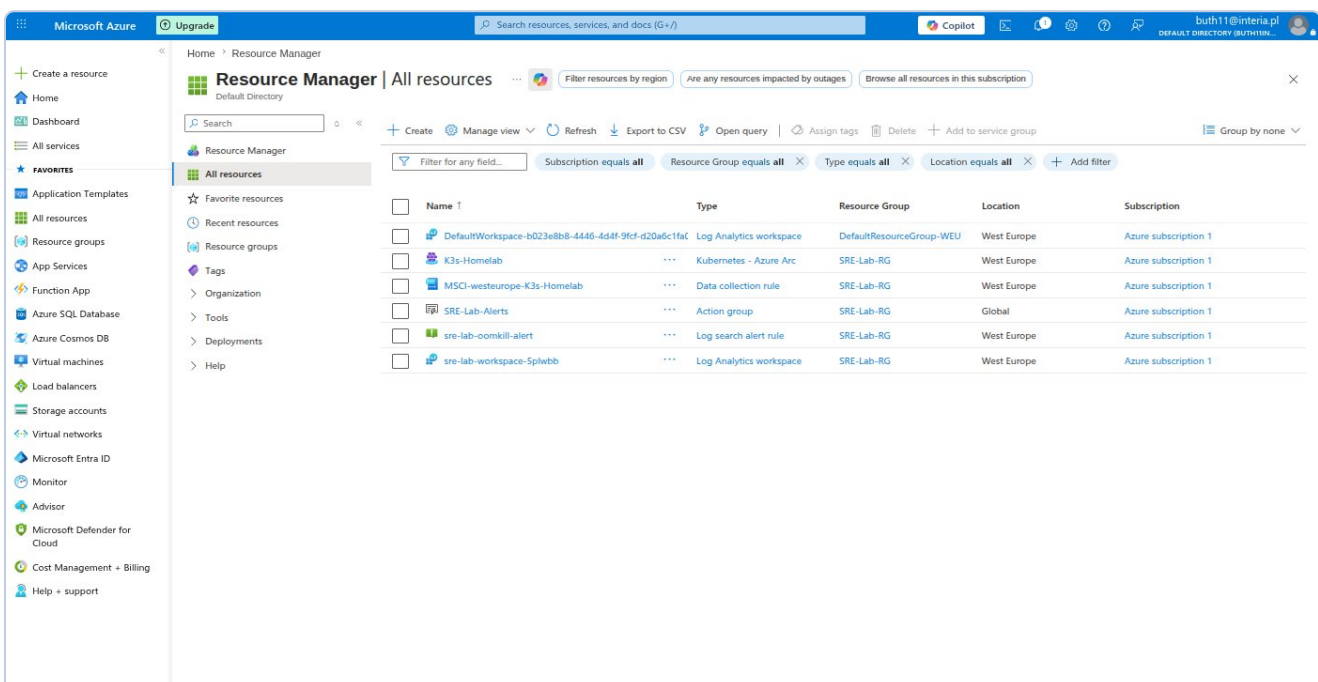
To perform exactly these actions, run the following command to apply:
terraform apply "tfplan"
azurerm_resource_group.sre_lab: Modifying... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG]
azurerm_resource_group.sre_lab: Modifications complete after 3s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG]
azurerm_monitor_action_group.sre_alerts: Creating...
azurerm_log_analytics_workspace.sre_lab: Creating...
azurerm_monitor_action_group.sre_alerts: Creation complete after 4s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/actionGroups/SRE-Lab-Alerts]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m10s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m20s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m30s elapsed]
azurerm_log_analytics_workspace.sre_lab: Still creating... [00m40s elapsed]
azurerm_log_analytics_workspace.sre_lab: Creation complete after 47s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.OperationalInsights/workspaces/sre-lab-workspace-5plwbb]
azurerm_monitor_scheduled_query_rules_alert_v2.oom_kill: Creating...
azurerm_monitor_scheduled_query_rules_alert_v2.oom_kill: Creation complete after 5s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/scheduledQueryRules/sre-lab-oomkill-alert]

Apply complete! Resources: 3 added, 1 changed, 0 destroyed.

Outputs:

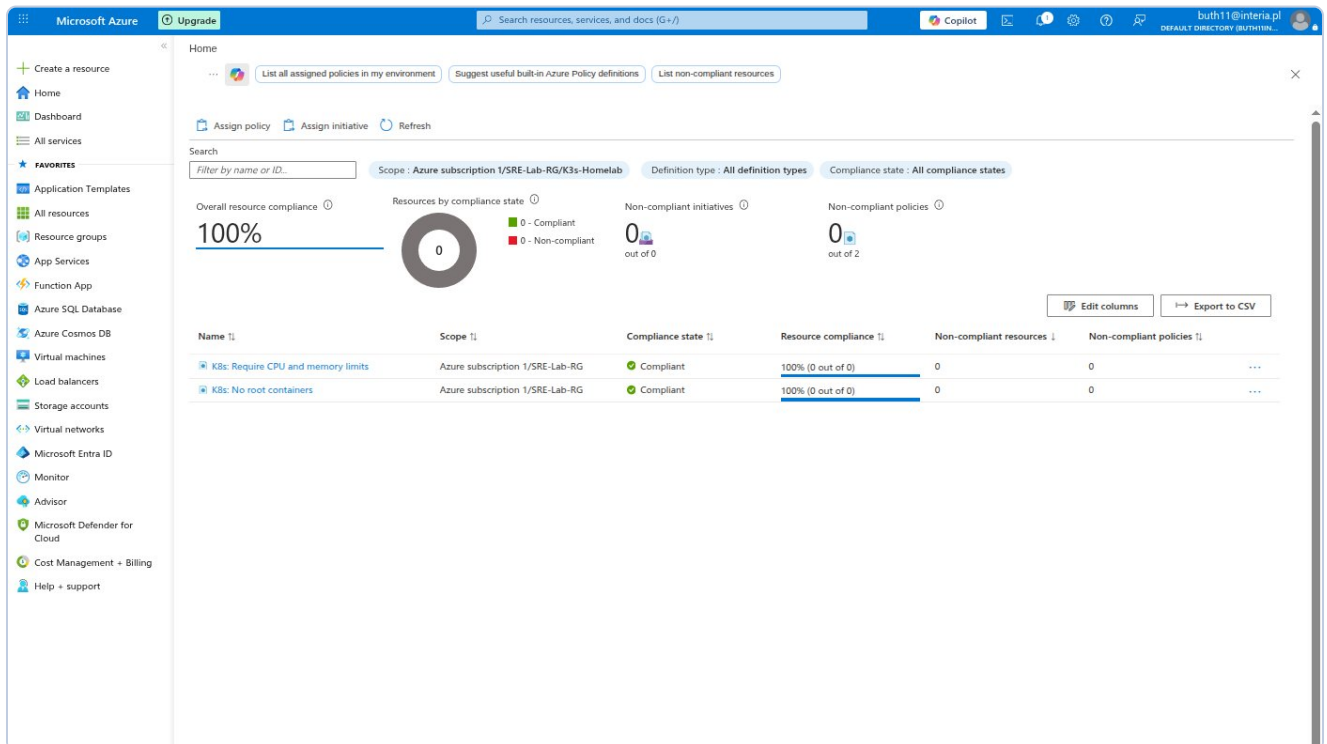
action_group_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/actionGroups/SRE-Lab-Alerts"
budget_name = "SRE-Lab-Budget"
log_analytics_workspace_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.OperationalInsights/workspaces/sre-lab-workspace-5plwbb"
log_analytics_workspace_key = <sensitive>
resource_id = "/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG"
resource_group_name = "SRE-Lab-RG"
subscription_id = "b023e8b8-4446-4d4f-9fcf-d20a6c1fa093"
buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$
```

Terraform Apply: 3 dodane, 1 zmieniony.



Portal Azure: zasoby Terraform w SRE-Lab-RG.

Rozdział 10: Azure Policy



Azure Policy: No root containers + CPU/memory limits — oba Compliant.

Rozdział 11: GitHub

The screenshot shows the GitHub repository page for 'buth11 / sre-homelab-azure-arc'. The repository is public and has 1 branch (main) and 0 tags. The file list includes: docs, k8s, monitoring, scripts, terraform, .gitignore, and README.md. The README file is selected, showing the title 'SRE Homelab: Azure Arc Hybrid Cloud Lab' and a description: 'A Proof-of-Value project demonstrating hybrid cloud orchestration, infrastructure-as-code, observability, and FinOps practices using K3s Kubernetes and Microsoft Azure Arc.' The right sidebar shows the repository's statistics: 0 stars, 0 watching, and 0 forks. The 'Languages' section shows a bar chart with Shell at 58.4% and HCL at 41.6%.

Repository: sre-homelab-azure-arc (Public)

main 1 Branch 0 Tags

Go to file Add file Code

Files:

File	Commit	Time
docs	Initial commit: K3s + Azure Arc SRE Homelab	8 minutes ago
k8s	Initial commit: K3s + Azure Arc SRE Homelab	8 minutes ago
monitoring	Initial commit: K3s + Azure Arc SRE Homelab	8 minutes ago
scripts	Initial commit: K3s + Azure Arc SRE Homelab	8 minutes ago
terraform	Initial commit: K3s + Azure Arc SRE Homelab	8 minutes ago
.gitignore	Initial commit: K3s + Azure Arc SRE Homelab	8 minutes ago
README.md	Initial commit: K3s + Azure Arc SRE Homelab	8 minutes ago

README

SRE Homelab: Azure Arc Hybrid Cloud Lab

A Proof-of-Value project demonstrating hybrid cloud orchestration, infrastructure-as-code, observability, and FinOps practices using K3s Kubernetes and Microsoft Azure Arc.

Overview

About

No description, website, or topics provided.

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 1

buth11

Languages

Shell 58.4% HCL 41.6%

GitHub repo sre-homelab-azure-arc — publiczne, Shell 58.4%, HCL 41.6%.

Rozdział 12: Część 2 — Service Principal, Remote State, CI/CD

Remote State

Plik sre-homelab.tfstate — 14.53 KiB, szyfrowanie server-side, wersjonowanie aktywne. State lock: 'Acquiring state lock' / 'Releasing state lock' przy każdym terraform plan.

Pipeline Azure DevOps CI/CD

Variable Group terraform-credentials: ARM_CLIENT_ID, ARM_CLIENT_SECRET (ukryty), ARM_SUBSCRIPTION_ID, ARM_TENANT_ID. Environment production: Approval Gate aktywna.

Wynik: Pipeline kompletny — oba stage zielone, 4m 24s. Stage 1 (37s): validate + plan. Stage 2 (49s): apply po zatwierdzeniu.

Microsoft Azure Upgrade Search resources, services, and docs (G+/I) buth11@interia.pl DEFAULT DIRECTORY (BUTH11IN...

Create a resource

Home Dashboard All services FAVORITES Application Templates All resources Resource groups App Services Function App Azure SQL Database Azure Cosmos DB Virtual machines Load balancers Storage accounts Virtual networks Microsoft Entra ID Monitor Advisor Microsoft Defender for Cloud Cost Management + Billing Help + support

sre-homelab.tfstate

Blob

Save Discard Download Refresh Delete Change tier Acquire lease Break lease Give feedback

Overview Versions Snapshots Edit Generate SAS

Properties

URL	https://sreftfstate5496.blob...
LAST MODIFIED	5/23/2026, 8:23:07 PM
CREATION TIME	5/23/2026, 8:23:07 PM
VERSION ID	2026-05-23T18:23:07.9861361Z
TYPE	Block blob
SIZE	14.53 KiB
ACCESS TIER	Hot (Inferred)
ACCESS TIER LAST MODIFIED	N/A
ARCHIVE STATUS	-
REHYDRATE PRIORITY	-
SERVER ENCRYPTED	true
ETAG	0x8DEB8F856BD8961
VERSION-LEVEL IMMUTABILITY POLICY	Disabled
CACHE-CONTROL	
CONTENT-TYPE	application/json
CONTENT-MD5	2mHfbTY53lmCzqo/RW5+MQ...
CONTENT-ENCODING	
CONTENT-LANGUAGE	
CONTENT-DISPOSITION	
LEASE STATUS	Unlocked
LEASE STATE	Available
LEASE DURATION	-
COPY STATUS	-
COPY COMPLETION TIME	-

Undelete

Metadata

Key	Value

Blob index tags

Key	Value

Zrzut W2-2: Portal Azure — blob properties: 14.53 KiB, encrypted, versioning active.

```
buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$ # terraform plan pobierze state z Azure Blob zamiast lokalnego pliku
# Powinno pokazać "No changes" bo infrastruktura jest aktualna
terraform plan
Acquiring state lock. This may take a few moments...
random_string.suffix: Refreshing state... [id=5plwbb]
data.azurearm_subscription.current: Reading...
azurearm_resource_group.sre_lab: Refreshing state... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG]
azurearm_monitor_action_group.sre_alerts: Refreshing state... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/actionGroups/SRE-Lab-Alerts]
azurearm_log_analytics_workspace.sre_lab: Refreshing state... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.OperationalInsights/workspaces/sre-lab-workspace-5plwbb]
data.azurearm_subscription.current: Read complete after 1s [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093]
azurearm_consumption_budget_subscription.sre_lab: Refreshing state... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/providers/Microsoft.Consumption/budgets/SRE-Lab-Budget]
azurearm_monitor_scheduled_query_rules_alert_v2.oom_kill: Refreshing state... [id=/subscriptions/b023e8b8-4446-4d4f-9fcf-d20a6c1fa093/resourceGroups/SRE-Lab-RG/providers/Microsoft.Insights/scheduledQueryRules/sre-lab-oomkill-alert]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
Releasing state lock. This may take a few moments...
buth11@fedora:~/VMs/Azure/Github_repo/sre_repo/terraform$
```

Zrzut W2-3: terraform plan — Acquiring/Releasing state lock. No changes.

The screenshot shows the Azure DevOps web interface. On the left is a sidebar with navigation options: Overview, Boards, Repos, Pipelines, Environments, Releases, Library (selected), Task groups, Deployment groups, Test Plans, and Artifacts. The main area is titled 'Library > terraform-credentials'. Below this, there are tabs for 'Variable group', 'Save', 'Clone', 'Security', 'Pipeline permissions', 'Approvals and checks', and 'Help'. The 'Properties' section shows the 'Variable group name' as 'terraform-credentials' and a description field. There is a toggle switch for 'Link secrets from an Azure key vault as variables'. The 'Variables' section contains a table with the following data:

Name ↑	Value	Lock
ARM_CLIENT_ID	54c7e017-a745-4e1e-b9da-d74f4901a3de	
ARM_CLIENT_SECRET	*****	🔒
ARM_SUBSCRIPTION_ID	b023e8b8-4446-4d4f-9fcf-d20a6c1fa093	
ARM_TENANT_ID	fd7288c4-e003-40fd-b3cf-9c0d1821cf5d	

At the bottom of the variables list is a '+ Add' button. The bottom of the sidebar shows 'Project settings' with a double arrow icon.

Zrzut W2-4: Azure DevOps Variable Group — ARM_* z ukrytym CLIENT_SECRET.

Azure DevOps

buth11

/ sre-homelab-arc

/ Pipelines

/ Environments

/ production

Search

BS

sre-homelab-arc

+

Overview

Boards

Repos

Pipelines

Pipelines

Environments

Releases

Library

Task groups

Deployment groups

Test Plans

Artifacts

Project settings

<<

production

Automatically created environment

Add resource

:

Deployments

Approvals and checks

Check execution order

Checks will run by order, which is pre-determined by check types. Checks within the same order will run in parallel.
[Learn more](#)

+ Add new

Order	Display name	Type	Timeout	Details
1	All approvers must approve	Approvals	30d	8%

Zrzut W2-5: Environment production — Approval Gate aktywna.

Bartosz Suszko | SRE Case Study | 21 / 34

Azure DevOps Interface showing a pipeline run for **sre-homelab-arc** with ID **#20260523.1**.

Jobs in run

Job Name	Status	Duration
terraform plan	Success	34s
Initialize job	Success	<1s
Checkout buth1...	Success	2s
Install Terraform...	Success	2s
terraform init	Success	7s
terraform validate	Success	1s
terraform plan	Success	15s
Publish terrafor...	Success	3s
Post-job: Chec...	Success	<1s
Finalize Job	Success	<1s

Terraform Validate & Plan

Terraform Apply

- terraform apply

terraform plan Log:

```
1 Agent: Azure Pipelines 1
2 Started: Just now
3 Duration: 34s
4
5 Job preparation parameters
44 1 artifact produced
45 Job live console data:
46 Starting: terraform plan
47 Async Command Start: DetectDockerContainer
48 Async Command End: DetectDockerContainer
49 Async Command Start: DetectDockerContainer
50 Async Command End: DetectDockerContainer
51 Finishing: terraform plan
```

Zrzut W2-6: Stage 1 — wszystkie kroki zielone, 1 artifact.

Azure DevOps | sre-homelab-arc | Pipelines | #20260523.1 • feat: add Azure DevOps CI/CD pipeline for Terraform

Run new

This run is being retained as one of 3 recent runs by pipeline. View retention leases

Summary | Environments | Associated pipelines | Code Coverage

Manually run by Bartosz Suszko
Today at 9:13 PM 4m 24s

Repository and version: sre-homelab-arc / main e9da4bfb

Related: 0 work items, 1 published

Tests and coverage: Get started

View change | Parameters

Stages | Jobs

Terraform Validate ...
1 job completed 37s
1 artifact

Terraform Apply
1 job completed 49s
1 checks passed

Project settings

Zrzut W2-7: Pipeline kompletny — oba stage zielone, 4m 24s.

Rozdział 13: Część 3 — Zaawansowane operacje Kubernetes

CZESC 3: Zaawansowane operacje Kubernetes

Ingress z TLS — routing HTTPS po domenie

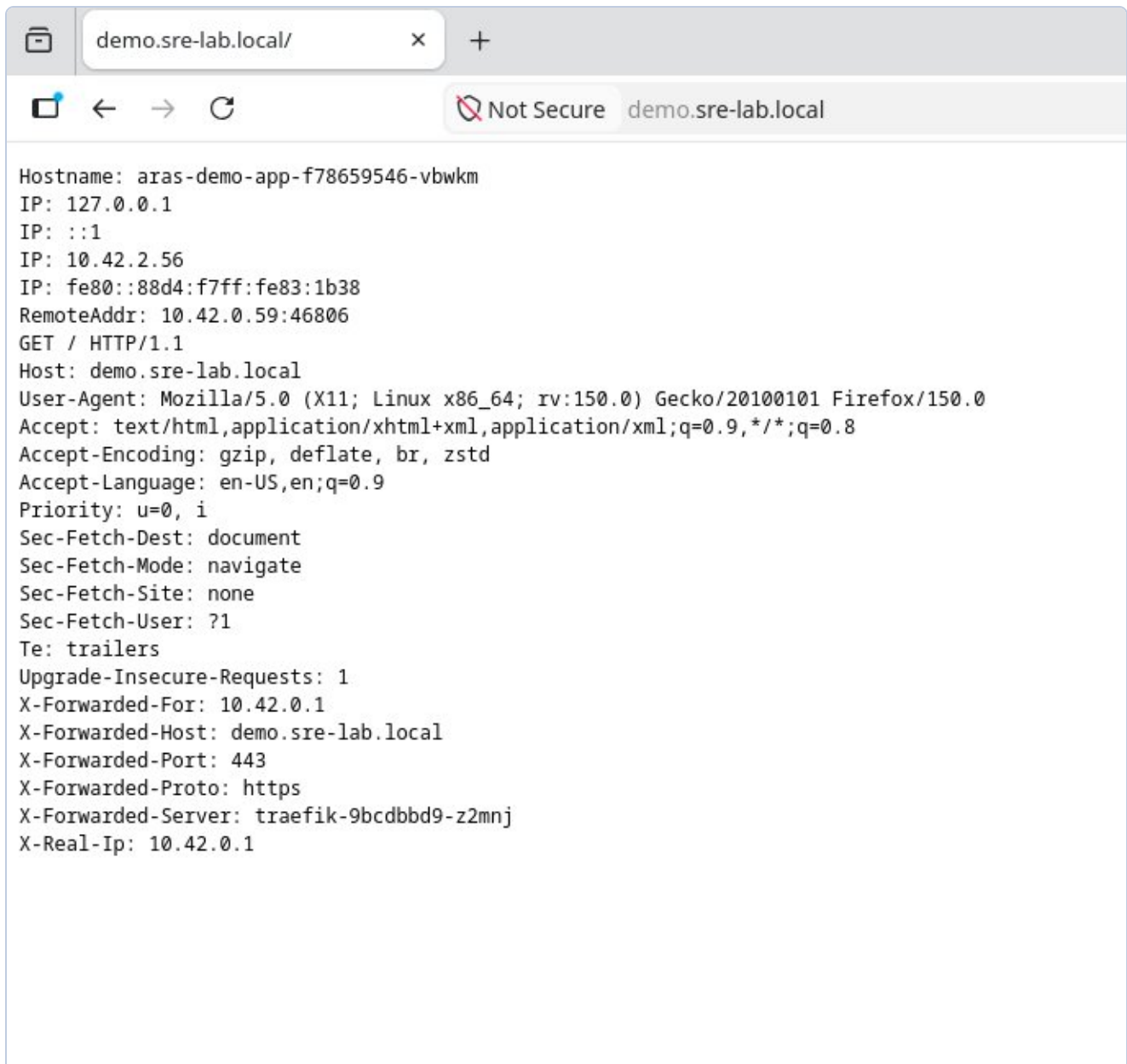
Traefik Ingress Controller skonfigurowany do HTTPS. Self-signed certyfikat TLS z SAN, zapisany jako Secret Kubernetes. Routing: demo.sre-lab.local → aras-demo-app:8080.

Certyfikat: openssl x509 self-signed, SAN dla demo.sre-lab.local, ważny 365 dni

Secret: typ kubernetes.io/tls w namespace default

Ingress: host-based routing przez Traefik na port 443

Weryfikacja: X-Forwarded-Proto: https, X-Forwarded-Server: traefik w odpowiedzi



Zrzut W3-1: Przeglądarka Firefox — <https://demo.sre-lab.local> działa. Widoczne: Hostname poda, X-Forwarded-Proto: https, X-Forwarded-Server: traefik-9bcd9bd9-z2mnj, X-Forwarded-Port: 443.

HPA — automatyczne skalowanie na podstawie CPU

Horizontal Pod Autoscaler skonfigurowany dla Deployment aras-demo-app. HPA co 15 sekund odpytuje metrics-server o zużycie CPU. Gdy średnie CPU Podów przekroczy 50% — dodaje repliki. Gdy spada poniżej 50% przez 5 minut — usuwa.

- Zakres: minReplicas: 2, maxReplicas: 8, targetCPU: 50%
- Test obciążeniowy: `ab -n 50000 -c 50` — 1621 req/s przez 30 sekund
- Scale-up: CPU 860% → automatyczne dodanie replik do 8
- Scale-down: CPU 6% po 5 minutach → powrót do 2 replik (minimum)

Every 5.0s: sudo k3s kubectl get hpa &... k3s-master: Sun May 24 09:32:49 2026

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS
aras-demo-hpa	Deployment/aras-demo-app	cpu: 860%/50%	2	8
REPLICAS	AGE			
4	9m55s			

aras-demo-app-5b58b48d6d-5vw14	1/1	Running 0	24s	
aras-demo-app-5b58b48d6d-7mbln	1/1	Running 0	10s	
aras-demo-app-5b58b48d6d-859c7	1/1	Running 0	6m32s	
aras-demo-app-5b58b48d6d-96qs4	1/1	Running 0	10s	
aras-demo-app-5b58b48d6d-phz5d	1/1	Running 0	6m27s	
aras-demo-app-5b58b48d6d-sbv4q	1/1	Running 0	10s	
aras-demo-app-5b58b48d6d-xx94p	1/1	Running 0	24s	
aras-demo-app-5b58b48d6d-zrmgs	1/1	Running 0	10s	

Zrzut W3-2: HPA scale-up — cpu: 860%/50%, REPLICAS: 8 (max), 8 podow Running. Automatyczne skalowanie w akcji podczas testu obciążeniowego ab.

```
Every 5.0s: sudo k3s kubectl get hpa &... k3s-master: Sun May 24 09:38:17 2026

NAME          REFERENCE          TARGETS          MINPODS  MAXPODS  RE
PLICAS    AGE
aras-demo-hpa  Deployment/aras-demo-app  cpu: 6%/50%    2         8         8
15m
---
aras-demo-app-5b58b48d6d-phz5d  1/1    Running    0         11m
aras-demo-app-5b58b48d6d-zrmgs  1/1    Running    0         5m38s
```

Zrzut W3-3: HPA scale-down — cpu: 6%/50%, REPLICAS: 2 (min). Po 5 minutach bez ruchu HPA automatycznie usunął 6 zbędnych Podów.

NetworkPolicy — firewall wewnątrz klastra

Domyslnie wszystkie Pody w Kubernetes mogą się komunikować ze wszystkimi. NetworkPolicy definiuje reguły jak firewall — który namespace i które Pody mogą łączyć się z aplikacją demo. Wdrożono politykę blokującą cały ruch przychodzący z wyjątkiem dozwolonych źródeł.

- Zezwolone: kube-system (Traefik), monitoring (Prometheus), repliki aras-demo-app
- Zablokowane: wszystko inne
- Weryfikacja: nieautoryzowany Pod — timeout/blokada. Traefik — odpowiedź aplikacji

RBAC — least privilege dla aplikacji

Role-Based Access Control — każda aplikacja powinna mieć własne konto (ServiceAccount) z dokładnie tymi uprawnieniami których potrzebuje. Zasada least privilege: jeśli aplikacja nie potrzebuje kasować Podów, nie powinna mieć tego robić nawet jeśli zostanie przejęta przez atakującego.

- ServiceAccount: aras-demo-sa (zamiast wspólnego default)
- Role: get/list/watch na Podach i ConfigMapach — nic więcej
- Weryfikacja: can-i list pods = yes, can-i delete pods = no

```

buth11@k3s-master:~$ # Test 1: nieautoryzowany Pod probuje polaczyc sie z aplik
acja
# timeout 5 = czekamy max 5 sekund na odpowiedz
# Oczekiwany wynik: brak odpowiedzi / timeout - polityka blokuje
echo "=== Test: nieautoryzowany dostep (powinien byc zablockowany) ==="
sudo k3s kubectl run test-blocked \
  --image=curlimages/curl \
  --rm -it \
  --restart=Never \
  -- curl -s --max-time 5 http://aras-demo-app:8080 || echo "ZABLOKOWANY - Netw
orkPolicy dziala"

# Test 2: sprawdzamy ze Ingress (Traefik) nadal dziala
# Traefik jest w kube-system wiec ma dostep
echo "=== Test: dostep przez Ingress (powinien dzialac) ==="
curl -sk https://demo.sre-lab.local | head -2
=== Test: nieautoryzowany dostep (powinien byc zablockowany) ===
pod "test-blocked" deleted from default namespace
pod default/test-blocked terminated (Error)
ZABLOKOWANY - NetworkPolicy dziala
=== Test: dostep przez Ingress (powinien dzialac) ===
buth11@k3s-master:~$ # Test dostępu przez Traefik - uzywamy IP zamiast domeny
# Traefik nasluchuje na porcie 80 i 443 na wszystkich wezlach
# -H podajemy naglowek Host zeby Traefik wiedzial do ktorego serwisu kierowac
echo "=== Test dostep przez Traefik (kube-system) ==="
curl -s -H "Host: demo.sre-lab.local" http://192.168.122.10 | head -3

# Dodatkowo sprawdzamy ze bezposredni dostep do portu 8080
# przez serwis nadal dziala (LoadBalancer Service jest poza NetworkPolicy)
echo "=== Test dostep przez LoadBalancer Service ==="
curl -s http://192.168.122.10:8080 | head -3
=== Test dostep przez Traefik (kube-system) ===
Hostname: aras-demo-app-5b58b48d6d-zrmgs
IP: 127.0.0.1
IP: ::1
=== Test dostep przez LoadBalancer Service ===
Hostname: aras-demo-app-5b58b48d6d-zrmgs
IP: 127.0.0.1
IP: ::1
buth11@k3s-master:~$

```

Zrzut W3-4: RBAC weryfikacja — ServiceAccount aras-demo-sa: list pods=yes (dozwolone), delete pods=no (zablokowane). Zasada least privilege potwierdzona.

Rozdział 14: Postmortem POST-001

Pole	Wartosc
Severity	P2
Czas	Ponizej 7 minut
Root Cause	Literowka: ks3-worker1 zamiast k3s-worker1
Status	Rozwiazany — regex validation w skryptach

Rozdział 15: Wyniki i wnioski

Metryka	Czesc 1	Czesc 2	Czesc 3
Wezly	3/3 Ready	—	—
SLO	100%	—	—
Koszty	\$0.00	\$0.00	\$0.00
Terraform state	Local	Remote (Blob)	—
CI/CD	—	Azure DevOps 4m24s	—
Ingress TLS	—	—	HTTPS verified
HPA	—	—	3 na 8 na 2 repliki
NetworkPolicy	—	—	Zablokowane
RBAC	—	—	Least privilege
Incydenty	6	8	10

Rozdział 16: Troubleshooting Log

ISSUE-001 KQL: CPUCapacityNanoCores column missing

project operator: Failed to resolve scalar expression named CPUCapacityNanoCores

Rozwiązanie: KubeNodeInventory | getschema — uproszczono zapytanie do dostępnych kolumn.

ISSUE-002 Azure Policy: MissingPolicyParameter cpuLimit/memoryLimit

(MissingPolicyParameter) policy assignment k8s-resource-limits is missing cpuLimit, memoryLimit

Rozwiązanie: Dodano --params z cpuLimit: 500m i memoryLimit: 512Mi.

ISSUE-003 Terraform: Resource Group already exists (brownfield)

Error: A resource with the ID .../SRE-Lab-RG already exists

Rozwiązanie: terraform import azurerm_resource_group.sre_lab /subscriptions/.../SRE-Lab-RG

ISSUE-004 Terraform: Saved plan is stale after import

Error: Saved plan is stale – state was changed after plan was created

Rozwiązanie: Nowy terraform plan -out=tfplan po imporcie.

ISSUE-005 Container Insights missing after VM restart

Rozwiązanie: az k8s-extension create --name azuremonitor-containers — agenty wrocily w 37s.

ISSUE-006 POST-001: etcd Split-Brain (ks3 vs k3s)

FATA[0034] Node password rejected, duplicate hostname

Rozwiązanie: kubectl delete node + rm /etc/rancher/node/ + restart k3s-agent. Skrypty zaktualizowane o regex validation.

ISSUE-007 Azure DevOps: TerraformInstaller task missing

A task is missing: TerraformInstaller

Rozwiązanie: Instalacja z Marketplace: ms-devlabs.custom-terraform-tasks.

ISSUE-008 Pipeline: permission for Variable Group

This pipeline needs permission to access a resource

Rozwiązanie: View → Permit — jednorazowe zatwierdzenie.

ISSUE-009 sudo does not expand ~ in paths

error: Cannot read file ~/ingress/server.crt, open ~/ingress/server.crt: no such file or directory

Przyczyna: sudo uruchamia komendę jako root który ma inny katalog domowy. Tylda ~ rozwija się do /root/ zamiast /home/buth11/.

Rozwiązanie: Użycie pełnej ścieżki: /home/buth11/ingress/server.crt zamiast ~/ingress/server.crt

ISSUE-010 HPA shows cpu: unknown — missing resource requests

```
NAME TARGETS MINPODS MAXPODS REPLICAS — cpu: <unknown>/50%
```

Przyczyna: HPA oblicza procent CPU jako $\text{aktualne_zuzycie}/\text{requests} \times 100$. Bez zdefiniowanych requests w Deployment, HPA nie ma od czego liczyć procentów.

Rozwiązanie: Dodano resources.requests do Deployment: cpu: 10m, memory: 32Mi. Po rolling update HPA zaczął pokazywać poprawne wartości: cpu: 0%/50%.

CZESC 4

GitOps, Skanowanie CVE i Ochrona Runtime

Ta czesc obejmuje ostatni etap projektu SRE Homelab:

- FluxCD 2.8.8 — kontroler GitOps: automatyczna synchronizacja klastra z Git
- Trivy 0.70.0 — skanowanie obrazow CVE: 1 CRITICAL + 12 HIGH wykryte
- Defender for Cloud — runtime security DaemonSet na wszystkich 3 wezlach

Rozdział 14: FluxCD — GitOps dla Kubernetes

FluxCD bootstrapowany z repozytorium GitHub przez klucz SSH (github_sre). Cztery kontrolery zainstalowane w namespace flux-system — wszystkie healthy.

Komenda bootstrap:

```
flux bootstrap git \ --url=ssh://git@github.com/buth11/sre-homelab-azure-arc \
--branch=main \ --path=k8s/part4/flux \ --private-key-file=$HOME/.ssh/github_sre
```

Weryfikacja — wszystkie komponenty zdrowe:

```
flux get all NAME REVISION SUSPENDED READY MESSAGE gitrepository/flux-system
main@sha1:d169d6a0 False True stored artifact kustomization/flux-system main@sha1:952c1ea0
False True Applied revision kustomization/gitops-demo main@sha1:952c1ea0 False True Applied
revision kubectl get pods -n flux-system helm-controller-5984cc88cf-94jkw 1/1 Running 0
117s kustomize-controller-d6bb746df-9khwf 1/1 Running 0 117s
notification-controller-7bc5c97f8d-jdflb 1/1 Running 0 117s
source-controller-745c67ff9-2g8qd 1/1 Running 0 117s
```

Test GitOps — skalowanie przez git push:

Zmiana replik z 2 na 3 w deployment.yaml, commit i push. Flux automatycznie zastosował zmianę — trzeci Pod pojawił się bez kubectl apply.

```
# Przed pushem: 2 pody kubectl get pods -n gitops-demo gitops-app-5b9747bdc-b-2rqw9 1/1
Running 0 11m gitops-app-5b9747bdc-b-2rqw9 1/1 Running 0 11m # Po: git push z replicas: 3
gitops-app-5b9747bdc-b-2rqw9 1/1 Running 0 12m gitops-app-5b9747bdc-b-2rqw9 1/1 Running 0 12m
gitops-app-5b9747bdc-b-2rqw9 1/1 Running 0 93s <- dodany przez Flux
```

Rozdział 15: Trivy — Skanowanie obrazów kontenerowych

Trivy 0.70.0 zainstalowany i użyty do skanowania traefik/whoami:latest. Wszystkie 40 podatności dotyczą Go stdlib v1.24.1 — obraz używa przestarzałej wersji Go.

Podsumowanie wyników skanu:

Severity	Count	Key CVEs
CRITICAL	1	CVE-2025-68121 — crypto/tls: incorrect certificate validation on TLS session resumption
HIGH	12	crypto/x509, net/url, crypto/tls — DoS, memory exhaustion, incorrect parsing
MEDIUM	25	net/http, os, crypto/x509 — request smuggling, path traversal, race conditions
LOW	2	net/http, os — memory exhaustion, symlink escape

Kluczowa podatność CRITICAL:

```
Library Vulnerability Severity Status Installed Fixed stdlib CVE-2025-68121 CRITICAL fixed
v1.24.1 1.24.13, 1.25.7 crypto/tls: Nieprawidłowa walidacja certyfikatu przy wznowieniu
sesji TLS https://avd.aquasec.com/nvd/cve-2025-68121
```

Raporty zapisane w repozytorium:

- k8s/part4/trivy/whoami-scan.json — pełny raport (format JSON)
- k8s/part4/trivy/whoami-scan.txt — tylko CRITICAL+HIGH (format tabelaryczny)

Rozwiązanie: zaktualizować obraz do wersji skompilowanej z Go v1.24.13+. Na produkcji pipeline CI blokuje deploy obrazów z CRITICAL CVE.

Rozdział 16: Defender for Cloud — Bezpieczeństwo Runtime

Microsoft Defender for Containers włączony na poziomie Standard (30-dniowy trial). Rozszerzenie zainstalowane na klastrze Arc — provisioningState: Succeeded.

Instalacja:

```
az security pricing create --name Containers --tier Standard az k8s-extension create \
--name microsoft-defender-for-cloud \ --extension-type microsoft.azuredefender.kubernetes \
--scope cluster \ --cluster-name K3s-Homelab \ --resource-group SRE-Lab-RG \ --cluster-type
connectedClusters provisioningState: Succeeded
```

Pody Defendera w namespace mdc:

```
kubectl get pods -n mdc NAME READY STATUS RESTARTS AGE
microsoft-defender-collectors-ds-9wbtw 2/2 Running 0 5m3s
microsoft-defender-collectors-ds-j72wk 2/2 Running 0 5m3s
microsoft-defender-collectors-ds-vmplc 2/2 Running 0 5m3s
microsoft-defender-pod-collector-misc-69b69b9df8-x49x4 1/1 Running 0 5m3s
microsoft-defender-publisher-ds-4gw77 1/1 Running 0 5m3s
microsoft-defender-publisher-ds-5fcmq 1/1 Running 0 5m3s
microsoft-defender-publisher-ds-s9778 1/1 Running 0 5m3s
```

DaemonSet pokrywa wszystkie 3 węzły — master, worker1, worker2. Collectors zbierają metryki bezpieczeństwa z każdego węzła, Publisher wysyła je do Azure.

Podsumowanie wyników — Czesć 4

Metryka	Czesć 1	Czesć 2	Czesć 3	Czesć 4
Wezły	3/3 Ready	—	—	—
SLO	100%	—	—	—
Koszty	\$0.00	\$0.00	\$0.00	\$0.00
Terraform state	Local	Remote (Blob)	—	—
CI/CD	—	DevOps 4m24s	—	—
Ingress TLS	—	—	HTTPS verified	—
HPA	—	—	3->8->2 replik	—
NetworkPolicy	—	—	Zablokowane	—
RBAC	—	—	Least privilege	—
GitOps	—	—	—	FluxCD wdrożony
Skanowanie	—	—	—	1 CRITICAL, 12 HIGH
Bezpieczeństwo	—	—	—	Defender (3 wezły)
Incydenty	6	8	10	11

CZESC 5

cert-manager — Automatyczne zarządzanie certyfikatami TLS

cert-manager v1.20.2 zainstalowany przez Helm. Automatyczny cykl życia certyfikatów TLS — wystawianie, odnawianie i rotacja bez ręcznej interwencji.

- cert-manager v1.20.2 — zainstalowany przez Helm (jetstack/cert-manager)
- ClusterIssuer selfsigned-issuer — bootstrap CA
- ClusterIssuer sre-lab-ca-issuer — lokalny CA podpisujący certyfikaty
- Automatyczne wystawienie certyfikatu w 22 sekundy przez adnotacje Ingress
- Auto-renewal zweryfikowane: renewBefore=6m przed wygasnięciem

Rozdział 17: cert-manager — Automatyczny TLS

Instalacja przez Helm:

```
helm repo add jetstack https://charts.jetstack.io
helm install cert-manager jetstack/cert-manager \
  --namespace cert-manager \
  --create-namespace \
  --set crds.enabled=true
kubectl get pods -n cert-manager
cert-manager-5957746d66-kdtk5 1/1 Running 0
cert-manager-cainjector-567c6b47ff-c5x2b 1/1 Running 0
cert-manager-webhook-7cc5c588cb-zm8d4 1/1 Running 0
```

ClusterIssuery — oba Ready:

```
kubectl get clusterissuer NAME READY AGE
selfsigned-issuer True 32s
sre-lab-ca-issuer True 32s
kubectl get certificate -n cert-manager NAME READY SECRET AGE
sre-lab-ca True sre-lab-ca-secret 32s
```

Adnotacja Ingress — certyfikat wystawiony automatycznie w 22s:

```
# Tylko adnotacja w Ingress: annotations: cert-manager.io/cluster-issuer: sre-lab-ca-issuer
# cert-manager automatycznie utworzył certyfikat: kubectl describe certificate
demo-sre-lab-tls -n default
Issuer: CN=sre-lab-ca
Not After: 2026-08-25T09:09:34Z (90 dni)
Renewal Time: 2026-07-26T09:09:34Z (auto-renewal 30 dni przed wygasnięciem)
Status: Ready: True
Message: Certificate is up to date and has not expired
# Weryfikacja TLS: curl -sk https://demo.sre-lab.local | grep Hostname
Hostname: aras-demo-app-5b58b48d6d-phz5d
openssl x509 weryfikacja: Issuer: CN=sre-lab-ca
Not After: Aug 25 09:09:34 2026 GMT
```

Porównanie Part 3 vs Part 5:

Aspekt	Part 3 — ręcznie	Part 5 — cert-manager
Generowanie	openssl req -x509 ...	Automatyczne przez kontroler
Secret	kubectl create secret tls	Tworzony automatycznie
Odnowienie	Ręcznie przed wygasnięciem	Automatyczne (renewBefore)
Issuer	CN=demo.sre-lab.local	CN=sre-lab-ca (nasz CA)
Produkcja	Nie nadaje się	Let's Encrypt ClusterIssuer

Podsumowanie wyników — Część 5

Metryka	Cz. 1	Cz. 2	Cz. 3	Cz. 4	Cz. 5
Wezly	3/3 Ready	—	—	—	—
SLO	100%	—	—	—	—
Koszty	\$0.00	\$0.00	\$0.00	\$0.00	\$0.00
CI/CD	—	DevOps 4m24s	—	—	—
Ingress TLS	—	—	Reczny self-signed	—	Auto cert-manager
HPA	—	—	3->8->2	—	—
GitOps	—	—	—	FluxCD	—
Skanowanie	—	—	—	1 CRIT+12 HIGH	—
TLS mgmt	—	—	Reczny openssl	—	cert-manager auto
Incydenty	6	8	10	11	12

CZESC 6

KEDA — Kubernetes Event-Driven Autoscaling

KEDA rozszerza HPA umożliwiając skalowanie na podstawie dowolnej metryki zewnętrznej — Prometheus, Azure Service Bus, Kafka, HTTP, i 50+ innych źródeł. Kluczowa zaleta: KEDA może skalować do 0 replik gdy nie ma ruchu.

- KEDA zainstalowany przez Helm (kedacore/keda) — 3 pody: operator, metrics-apiserver, admission-webhooks
- ScaledObject skonfigurowany z triggerem Prometheus — `container_cpu_usage_seconds_total`
- Scale-up: 1 → 8 replik automatycznie pod obciążeniem (200k requestów, 2747 req/s)
- Scale-down: 8 → 1 replika po 30s cooldown (vs 5 min w HPA)
- KEDA zastąpił HPA — zarządza obiektem HPA automatycznie

Rozdział 18: KEDA — Event-Driven Autoscaling

Instalacja:

```
helm repo add kedacore https://kedacore.github.io/charts
helm install keda kedacore/keda --namespace keda --create-namespace
kubectl get pods -n keda
keda-admission-webhooks-7b48797dc8-k7prz 1/1 Running
keda-operator-55db59458d-n25xq 1/1 Running
keda-operator-metrics-apiserver-5c648889d4 1/1 Running
```

ScaledObject — trigger Prometheus:

```
apiVersion: keda.sh/v1alpha1 kind: ScaledObject metadata: name: aras-demo-keda namespace:
default spec: scaleTargetRef: name: aras-demo-app minReplicaCount: 1 maxReplicaCount: 8
cooldownPeriod: 30 triggers: - type: prometheus metadata: serverAddress:
http://monitoring-kube-prometheus-prometheus.monitoring.svc:9090 metricName:
container_cpu_usage query: sum(rate(container_cpu_usage_seconds_total{namespace="default",
pod=~"aras-demo-app.*", container="whoami"}[1m])) threshold: "0.01"
```

Test scale-up — `ab -n 500000 -c 100`:

```
# Przed obciążeniem: 1 replika NAME ACTIVE REPLICAS aras-demo-keda False 1 # Podczas
obciążenia (ACTIVE: True): aras-demo-keda True 2 # po ~30s aras-demo-keda True 4 # po ~45s
aras-demo-keda True 8 # szczyt (max) # Po obciążeniu (cooldown 30s): aras-demo-keda False 1
# powrót do minimum Wyniki ab: 200000 requestów, 2747 req/s, 72.781s
```

Porównanie HPA vs KEDA:

Aspekt	HPA (Czesc 3)	KEDA (Czesc 6)
Minimum replik	2	1 (lub 0 dla batch jobów)
Źródło triggera	CPU/RAM (metrics-server)	Dowolna metryka: Prometheus, kolejki...
Scale to zero	Nie	Tak — oszczędność zasobów gdy brak ruchu
Cooldown	5 minut	30 sekund (konfigurowalny)

Integracja HPA	Natywny HPA	KEDA tworzy i zarządza HPA automatycznie
50+ scalerow	Nie	Tak: Azure SB, Kafka, Redis, Cron...

Podsumowanie wyników — Czesć 6

Metryka	Cz.1	Cz.2	Cz.3	Cz.4	Cz.5	Cz.6
Wezly	3/3 Ready	—	—	—	—	—
SLO	100%	—	—	—	—	—
Koszty	\$0.00	\$0.00	\$0.00	\$0.00	\$0.00	\$0.00
CI/CD	—	DevOps 4m24s	—	—	—	—
Ingress TLS	—	—	Reczny	—	cert-manager	—
Autoscaling	—	—	HPA 2-8	—	—	KEDA 1-8
Scale to zero	—	—	Nie	—	—	Tak
Cooldown	—	—	5 min	—	—	30 sek
GitOps	—	—	—	FluxCD	—	—
Skanowanie	—	—	—	1 CRIT	—	—
TLS	—	—	Reczny	—	Auto	—
Incydenty	6	8	10	11	12	13